Критический анализ документации

Повторная попытка оплаты, смена способа оплаты

Дата отчёта: 27 июля 2026

Аудитор: Senior Technical Writer / QA Engineer

Контекст анализа

Основной документ

«Повторная попытка оплаты, смена способа оплаты» (Смена способа оплаты.pdf)

Сравниваемые документы

- Ordering — GET /payment-methods
- Payments — фильтрация RETRY_PAY (Internal payment-methods)
- Ordering — маппинг статусов Kafka (Смена способов оплаты)
- Ordering — /retry-pay-by-unlinked-card
- Ordering — /retry-pay-by-unlinked-sberpay
- Ordering — /retry-pay-by-linked-pmi
- Ordering — обработка /extrahold
- Payments — extrahold на HOLD_FAILED

Целевая аудитория

Разработчики (backend Ordering/Payments/Rubilnik, mobile) и тестировщики (E2E). Документ предполагает знание внутренней архитектуры (Kafka, OMS, Rubilnik, massage_collector), но ключевые API-контракты и UI-сценарии должны быть самодостаточными для реализации и тест-дизайна.

Таблица дефектов

Ниже приведён полный перечень выявленных дефектов. Критичность: High — блокирует разработку/тестирование; Med — риск ошибок; Low — улучшения.

#1 [High] Непротиворечивость (именование поля)

Фрагмент:

Основной: available_function (п. 4.1, 4.10–4.12); дочерние: available_functions (маппинг Kafka, retry-pay методы)

Почему это плохо:

Разные имена одного поля в БД, Kafka-логике и API приведут к расхождению контрактов и падению интеграционных тестов.

Рекомендация:

Зафиксировать единое имя во всех документах (например, available_function) и добавить таблицу маппинга «источник → поле ответа API».

#2 [High] Неполнота (заглушки вместо ТЗ)

Фрагмент:

Основной п. 4.3–4.6: «Требования: MS Ordering.API /retry-pay-by-unlinked-card» без логики

Почему это плохо:

В основном ТЗ отсутствует бизнес-логика ключевых эндпоинтов смены способа оплаты; разработчик/тестировщик вынужден искать дочерние документы.

Рекомендация:

В основном документе для каждого метода добавить preconditions, пошаговую логику, коды ответов, побочные эффекты. Либо явно указать ссылку на полное ТЗ с версией.

#3 [High] Непротиворечивость (HTTP-коды)

Фрагмент:

payment-methods: невалидный токен → 401, чужой заказ → 403; retry-pay-by-*: невалидный токен → 403, чужой заказ → 422

Почему это плохо:

Клиенты и автотесты не смогут единообразно обрабатывать ошибки авторизации/доступа в рамках одного user flow.

Рекомендация:

Унифицировать матрицу ошибок для всех public API Ordering в сценарии смены способа оплаты (401/403/422/424).

#4 [High] Непротиворечивость (логика extrahold)

Фрагмент:

Основной п. 6.2: разрешить HOLD_FAILED → EXTRA_HOLD_WAITING; Payments doc: «Если статус = HOLD_SUCCESS ИЛИ HOLD_FAILED» и «Если статус != HOLD_SUCCESS, то вернуть ошибку»

Почему это плохо:

Противоречие в одном разделе Payments делает реализацию неоднозначной: допускается ли extrahold после HOLD_FAILED или нет.

Рекомендация:

Переписать логику Payments extrahold однозначно: IF status IN (HOLD_SUCCESS, HOLD_FAILED) THEN proceed ELSE error.

#5 [High] Неполнота (бизнес-правила смены PMI)

Фрагмент:

Основной документ не описывает пороги attempt_count, ER034, ER025, error_code=116 для change_pmi vs retry_pay

Почему это плохо:

Ключевое условие «когда показывать смену способа оплаты» живёт только в документе маппинга Kafka.

Рекомендация:

В основной документ добавить decision table: error_code × attempt_count × payment_type × toggles → available_function.

#6 [High] Неполнота (тексты пушей)

Фрагмент:

Основной п. 4.8: «(НУЖЕН ТЕКСТ ПУША)»; маппинг: «ЖДЕМ КОНТРАКТ» для payment-reminder

Почему это плохо:

Невозможно реализовать и протестировать уведомления и таймерные джобы без финальных текстов и payload.

Рекомендация:

Добавить приложение с финальными message_title, message_text, параметрами push/deeplink и телом payment-reminder для сценария change_pmi.

#7 [High] Неполнота (инвалидация кэша)

Фрагмент:

Основной п. 4.2: «создать кэш»; payment-methods: TTL 30 мин; retry-pay: «очистить available_function»

Почему это плохо:

После смены способа оплаты кэш может отдавать устаревший список методов до 30 минут; сценарий не описан.

Рекомендация:

Описать события инвалидации кэша: смена PMI, HOLD_SUCCESS/FAILED, истечение retry_pay_deadline, отмена партиции.

#8 [Med] Непротиворечивость (nullable vs обязательное)

Фрагмент:

БД: available_function — Nullable; API (п. 4.10–4.12): поле обязательное

Почему это плохо:

Неясно, что возвращать, если status != WAIT_RETRY_PAY или поле NULL.

Рекомендация:

Указать: поле возвращается только при is_retry_pay=true и status=WAIT_RETRY_PAY; иначе — не включать в ответ или null с явным правилом.

#9 [Med] Непротиворечивость (enum vs массив)

Фрагмент:

БД/API: string enum (retry_pay | change_pmi); комментарий: «в перспективе пользователь сам будет выбирать»

Почему это плохо:

Неясно, может ли backend когда-либо вернуть оба сценария; mobile строит взаимоисключающий UI.

Рекомендация:

Явно зафиксировать: «в v1 всегда ровно одно значение»; multi-value — отдельная версия API.

#10 [Med] Непротиворечивость (коды ошибок)

Фрагмент:

Основной п. 4.13: при change_pmi на старом /retry-pay → 424; retry-pay-by-*: RETRY_PAY_CHANGE_PMI=false → 422

Почему это плохо:

Разные HTTP-коды для схожих бизнес-ошибок усложняют клиентскую обработку.

Рекомендация:

Согласовать: 424 = «дедлайн/заказ нельзя оплатить», 422 = «фича выключена/невалидный запрос». Зафиксировать в таблице.

#11 [Med] Неполнота (инкремент attempt_count)

Фрагмент:

Оценка: «Сделать attemt_cout +1» (CHEK-1078); в требованиях п. 4.1–4.13 логика инкремента не описана

Почему это плохо:

Тестировщик не знает, когда увеличивается счётчик: при Kafka-событии, при вызове retry, при неуспешном hold?

Рекомендация:

Добавить раздел «Жизненный цикл attempt_count»: триггеры, границы, связь с retry_pay/change_pmi.

#12 [Med] Точность (опечатки в идентификаторах)

Фрагмент:

attemt_cout, MS Orderind.API, payments_methods, PACKAGING (кириллическая «С»)

Почему это плохо:

Риск ошибок в коде, миграциях и SQL-фильтрах; поиск по кодовой базе не сработает.

Рекомендация:

Прогнать глоссарий, исправить во всех документах: attempt_count, PACKAGING, единое имя таблицы payment_methods.

#13 [Med] Непротиворечивость (регистр type)

Фрагмент:

Mobile: type == "BANKCARD"; Payments Internal: bankcard, orange_pay (lowercase); Ordering response enum: BANKCARD, ORANGE_PAY

Почему это плохо:

Mobile может не распознать тип способа оплаты при проксировании ответа Payments → Ordering.

Рекомендация:

Зафиксировать единый регистр в контракте Ordering public API (рекомендуется UPPER_SNAKE как в примере ответа).

#14 [Med] Неполнота (RETRY_PAY в payment-methods)

Фрагмент:

payment-methods: при WAIT_RETRY_PAY передаётся фильтр RETRY_PAY; mobile: «SBP/ADD_SBP — невозможен в кейсе»

Почему это плохо:

Нет явного правила фильтрации на стороне Ordering для UI (скрывать vs is_active=false).

Рекомендация:

Добавить: «для target=RETRY_PAY не возвращать SBP, ADD_SBP, INSTANT_SBP, PODELI»; указать поведение is_active.

#15 [Med] Неполнота (конфиг ER025)

Фрагмент:

П. 4.9: «15мин — конфигурируемое… *написать как будет называться конфиг»

Почему это плохо:

Нельзя настроить и протестировать отложенный пуш ER025 в разных средах.

Рекомендация:

Указать имя конфига, default, единицы (сек/мин), scope (per-tenant/per-env).

#16 [Med] Ясность (магические задержки)

Фрагмент:

Mobile: «обновить деталку с задержкой 10 сек» (3 раза)

Почему это плохо:

Нет критерия успеха: что если за 10 сек статус ещё не обновился? Flaky E2E.

Рекомендация:

Заменить на polling/retry с backoff до N сек или push/WebSocket-триггер; указать ожидаемый статус после обновления.

#17 [Med] Неполнота (fingerprint)

Фрагмент:

Mobile: «получить и отправить fingerprint из kaf»; linked-pmi: обязателен для ORANGE_PAY

Почему это плохо:

Неясный источник данных и момент получения; опечатка «kaf».

Рекомендация:

Указать SDK/метод получения fingerprint, формат, обязательность по платформам, поведение при отсутствии.

#18 [Med] Атомарность

Фрагмент:

Основной документ объединяет БД, 6+ API, Kafka, 2 джобы, Rubilnik, Payments, mobile в одном файле

Почему это плохо:

Сложно ревьюить, тестировать и отслеживать готовность по компонентам.

Рекомендация:

Разбить на отдельные спецификации с перекрёстными ссылками; в основном — только overview + матрица трассируемости.

#19 [Med] Неполнота (диаграммы)

Фрагмент:

Основной п. 3 «Схема взаимодействия» — страницы 2–8 пустые в текстовой выгрузке

Почему это плохо:

Без sequence diagram непонятен порядок вызовов: Ordering → Payments → Kafka → OMS → push.

Рекомендация:

Восстановить/вложить диаграмму с нумерацией шагов, синхронизированной с тест-кейсами.

#20 [Med] Неполнота (HOLD_FAILED + change_pmi для Сбер)

Фрагмент:

Маппинг Kafka: для HOLD_FAILED при RETRY_PAY_CHANGE_PMI=true описан в основном retry_pay; ветка change_pmi для Сбер — обрывается

Почему это плохо:

QA не сможет покрыть сценарий смены карты после неуспешного hold Сбера.

Рекомендация:

Дописать ветку change_pmi для HOLD_FAILED/extrahold failed с теми же порогами attempt_count.

#21 [Low] Форматирование

Фрагмент:

П. 7 mobile: «figma» без ссылки; дубли «Если Если»; разорванные условия retry_pay_deadline

Почему это плохо:

Ухудшает читаемость, повышает риск неверной реализации UI-условий.

Рекомендация:

Отформатировать условия в псевдокод/table; вставить рабочую ссылку на Figma.

#22 [Low] Неполнота (deeplink timing)

Фрагмент:

Mobile: «Триггер… пинговать схемы диплинков — на стороне разработки»

Почему это плохо:

Требование не зафиксировано; iOS/Android могут вести себя по-разному.

Рекомендация:

Описать минимум: когда вызывать GET /api/payments/v1/deeplink-ios-schemes, timeout, fallback при недоступности СБОЛ.

#23 [Low] Неполнота (push при смене PMI)

Фрагмент:

Mobile: «ОБСУДИТЬ НА ГРУММЕ НУЖЕН ЛИ… ОСОБЕННЫЙ ПУШ»

Почему это плохо:

Открытый вопрос в релизном ТЗ.

Рекомендация:

Закрыть решением: тип push, payload, deep link на bottom sheet.

#24 [Low] Неполнота (аналитика)

Фрагмент:

Оценка: «Продуктовая аналитика ????»

Почему это плохо:

Нет событий для продуктовой аналитики смены способа оплаты.

Рекомендация:

Добавить список analytics events или явно указать «вне скоупа».

#25 [Low] Неполнота (/confirm)

Фрагмент:

Основной п. 6.2: «/confirm доработать места где требуется успешный HOLD»

Почему это плохо:

Нет отдельной спецификации; неясно, какие endpoint'ы затронуты.

Рекомендация:

Список затронутых методов Payments с описанием новой логики поиска hold/extrahold.

#26 [Low] Полнота (права/тогглы)

Фрагмент:

Rubilnik: только «создать тогл RETRY_PAY_CHANGE_PMI»

Почему это плохо:

Нет матрицы: RETRY_PAY × RETRY_PAY_CHANGE_PMI × seller × deadline → поведение.

Рекомендация:

Добавить таблицу комбинаций тогглов и ожидаемого UX/API-ответа.

Сводка по критериям качества

Полнота: Низкая

Основной документ — index/epic: ключевые API (4.3–4.6) без логики; открытые «ЖДЕМ КОНТРАКТ», пустые пуши, нет правил attempt_count.

Точность и непротиворечивость: Критичные расхождения

available_function vs available_functions; 401/403/422; противоречие в extrahold; разный регистр type.

Ясность: Средняя

Много отсылок «как на чекауте», «логика без изменения», «на стороне разработки» без критериев.

Атомарность: Низкая

Один документ на 3 сервиса + mobile; детали размазаны по 8 PDF без единого глоссария.

Форматирование: Средняя

Таблицы в дочерних документах хорошие; в основном — рваный текст, битые условия в mobile-разделе.

Критичные расхождения «Основной ↔ Дочерние»

- Имя поля: available_function (основной, API GET orders) ≠ available_functions (Kafka, retry-pay).
- Авторизация: 401 vs 403 для невалидного токена между payment-methods и retry-pay-by-*.
- Владелец заказа: 403 vs 422.
- Extrahold на HOLD_FAILED: основной и Payments-требование конфликтуют внутри одного child-doc.
- Пороги смены PMI: есть только в маппинге Kafka, отсутствуют в основном ТЗ и mobile.
- Кэш payment-methods: создан в ТЗ, но жизненный цикл при смене PMI не описан.

Рекомендации для тестирования (приоритет)

- Переход retry_pay → change_pmi по attempt_count / error_code (ER034, ER025, 116).
- Негативные сценарии HTTP (401/403/422/424/409) на всех retry-эндпоинтах.
- Extrahold после HOLD_FAILED (после уточнения Payments-логики).
- Консистентность кэша payment-methods после успешной смены PMI.
- Mobile: истечение retry_pay_deadline при активной кнопке «Готово» (граничные кейсы не расписаны).

Примечание: тест-дизайн рекомендуется строить по дочерним API-спецификациям, а не только по основному PDF, до закрытия критичных дефектов документации.